



دانشگاه شهیدچمران اهواز

Shahid Chamran

University of Ahvaz

دانشکده مهندسی

گروه مهندسی برق

گزارش پروژه درسی سیستم دیجیتال 2

پروژه با میکروکنترلر AVR خانواده ATmega و کدنویسی با آتمل استدیو و شبیه سازی در پروتئوس

عنوان پروژه:

[تشخیص نشت گاز با استفاده از mq-135 و هشدار از طریق ارسال پیامک و

بازر و آل ای دی]

دانشجویان:

[علی یوسفی کیا 400655106 محسن پولاد 400655019 امیرحسین ژولا یوسفی

400655042]

نام استاد درس:

دکتر نوید علایی شینی

تاریخ تحویل گزارش :

خرداد 1403



چکیده

مدار طراحی شده با استفاده از ماژول mq-135 غلظت گاز متان و کربن دی اکسید هوا را اندازه گیری میکند. هرگاه غلظت گاز بیش از مقداری که از قبل به واسطه پتانسیومتر ها(یکی متصل به ماژول و دیگری متصل به (pA1) است) تعیین کردیم برسد، مدار ابتدا با روشن کردن LED و سپس ایجاد صدا با استفاده از بازر و درنهایت ارسال پیامک با استفاده از ماژول SIM-800 کاربر را از خطرات مطلع میکند. لازم به ذکر است که در تمام این مراحل غلظت گاز به طور لحظه ای توسط یک LCD کاراکتری 2 در 16 مانیتور میشود.

1- فصل اول

1 **سخت افزار سیستم**

1-1- انتخاب قطعات : 2

1-2- لیست قطعات : 2

1-3- نقشه سخت افزار پروژه 5

1-4 6

2- فصل دوم

8 **نرم افزار**

2-1 توضیحات خط به خط کد ها: 9

2-2 فلوچارت برنامه 16

3- فصل سوم نتایج

3-1 نتایج شبیه سازی در نرم افزار پروتئوس: 18

3-2 مراجع 19

فصل اول

سخت افزار سیستم

1-1- انتخاب قطعات :

ما در این پروژه به دنبال بستن یک مدار برای تشخیص غلظت گاز و جلوگیری از گازگرفتگی هستیم و به همین دلیل از ماژول MQ135 استفاده کردیم که به دود، CO₂، آمونیاک، سولفید و متان حساس است. برای پردازش کدها از میکروکنترلر ATMEGA 16 استفاده میکنیم که یک میکروکنترلر مناسب با توجه به نیازهای ما است و با حداکثر فرکانس 16 مگاهرتز کار میکند. برای ارسال پیامک ما از SIM800 استفاده کردیم با توجه به دیتاشیت محصول که ولتاژ باید در رنج 3.7 تا 4.4 باشد ما نیاز به یک کاهنده داریم و از ماژول مبدل LM2596 استفاده میکنیم. کلاک داخلی ATMEGA 16 پایین هست به همین دلیل ما نیاز به یک کلاک خارجی داریم و از یک کریستال 8 مگاهرتز باید استفاده کنیم تا کلاک مورد نیاز ما تامین شود.

1-2- لیست قطعات :

1- میکروکنترلر atmega 16

AVR ATmega16 یک میکروکنترلر 8 بیتی CMOS کم مصرف است که بر اساس معماری RISC تقویت شده ساخته شده است. AVR دارای 32 رجیستر با اهداف عمومی و یک مجموعه دستورالعمل غنی است. آنها مستقیماً به ALU متصل هستند و به دو رجیستر مستقل اجازه می دهند تا در یک دستورالعمل اجرا شده در یک چرخه ساعت دسترسی داشته باشند.

2- رله 5 ولت :

ماژول رله 5 ولت نوعی ماژول رله است که برای کار کردن به ورودی 5 ولت DC نیاز دارد. ماژول رله 5 ولت، یک ماژول رله تک یا چند کاناله است که با ولتاژ تریگر سطح پایین 5 ولت DC کار می کند. مانند بسیاری از رله های دیگر، ماژول رله 5 ولتی یک کلید الکترومغناطیسی است که با برق کار می کند و می توان از آن برای روشن یا خاموش کردن مدار استفاده کرد. مشخصات ماژول رله 5 ولتی بسته به سازنده متفاوت است.

3- ماژول sim 800

sim800 یک قطعه یا ماژول GSM/GPRS چهار بانده است که برای بازار جهانی طراحی شده و بر روی فرکانس های 1900/1800/900/850 MHz کار می کند. SIM800 دارای GPRS چند اسلات کلاس 12 و کلاس 10 است که از طرح های کدگذاری CS-1.2.3.4 پشتیبانی می کند. دستورات مخابراتی بر مبنی دستورات AT بوده و قابلیت ارسال و دریافت تماس، اس ام اس ((SMS و دیتا را داراست. این محصول با رنج ولتاژ کاری v3.4 تا v4.4 و تکنیک صرفه جویی در انرژی طراحی شده به طوری که مصرف جریان در حالت بیکاری به 0.55 میلی آمپر می رسد. SIM800 با پیکربندی کوچک 24*24*3 میلی متر می تواند ابعاد کوچکی را در پروژه ها برآورده کند.

4- مبدل LM2596

با استفاده از این ماژول می توان توسط یک مولتی ترن که بر روی ماژول وجود دارد ولتاژ ایده آل خود را تنظیم کنید. در بعضی از پروژه ها ولتاژی به غیر از ولتاژهای استاندارد مورد نیاز است یا اینکه ولتاژ منبعی که برای مدار وجود دارد بیشتر از حد ولتاژ مورد نیاز شما است

5- ماژول MQ135

سنسورهای گاز سری MQ از هیتر داخلی کوچک به همراه سنسور الکتروشیمیایی بهره می گیرند. این سنسورها نسبت به طیف گسترده ای از گازها حساس اند. و در خانه و دمای اتاق استفاده می شوند. سنسور MQ-135، سنسوری برای تشخیص کیفیت هوا می باشد. این سنسور قابلیت دود، CO₂، آمونیاک، سولفید و بنزن موجود در هوا را دارد. هیتر این سنسور از ولتاژ V5 استفاده می کند

LCD 16*2

دلیل نام گذاری به صورت 2×16 LCD به این خاطر است که دارای 16 ستون و 2 سطر است. ترکیبات زیادی مانند 1×8 , 2×8 , 2×10 , 1×16 و ... وجود دارد اما یکی از پر استفاده ترین آن ها * 16×1 LCD 2 است، بنابراین ما در اینجا از آن استفاده می کنیم

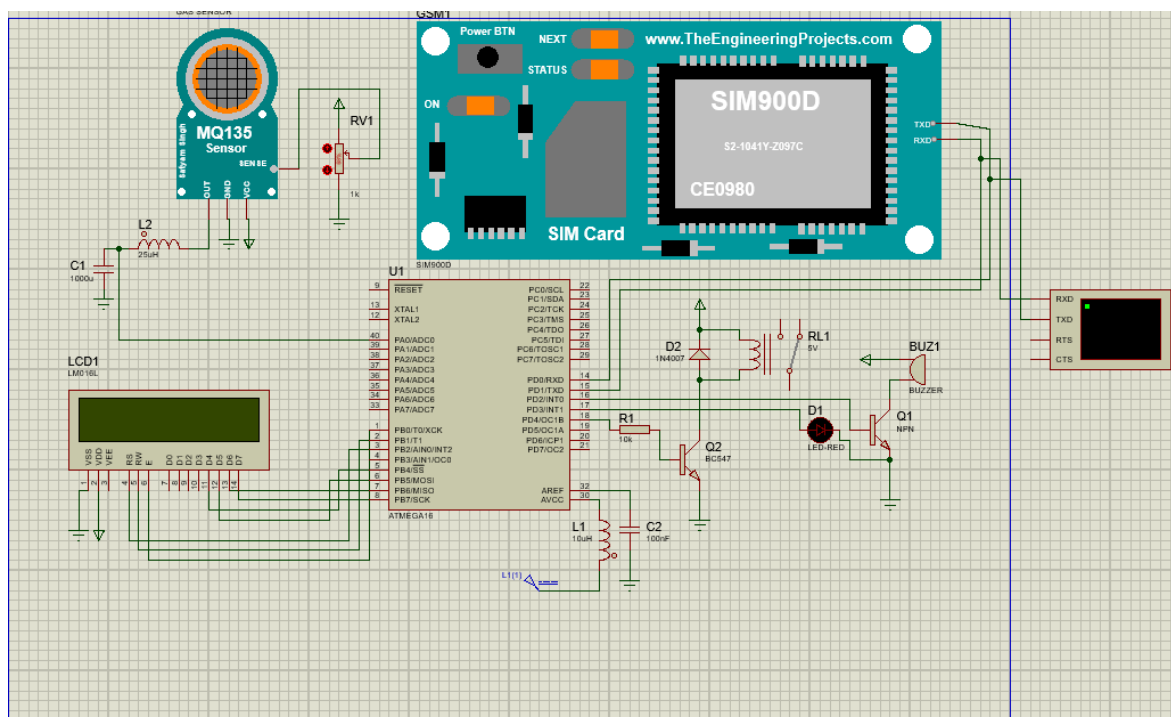
6- کریستال 8 مگاهرتز

کریستال ها یکی از مهم ترین و پرکاربردترین ابزارها در سیستم های دیجیتال به شمار می روند. به کمک این ابزار می توان میزان کلاک مورد نیاز آی سیس دیجیتال را فراهم نمود. به عنوان مثال میکروکنترلرها جهت راه اندازی و کار نیاز به منبع کلاک دارند. کریستال 8 مگاهرتز یک منبع کلاک و یا نوسان ساز مناسب جهت راه اندازی آی سی های میکروکنترلر است. به کمک این قطعه می توانید فرکانس 8 مگاهرتزی جهت راه اندازی قطعه دیجیتال ایجاد کنید. به کمک این کریستال می توان آی سی های میکروکنترلی نظیر ATMEGA32 و یا ATMEGA16 را راه اندازی نمود.

7- باز 5 ولت

بازر قطعه کوچک الکترونیکی است که همانند بلندگو انرژی الکتریکی را به صدا تبدیل می کند، اما با فرکانس مشخصی که باعث صدایی شبیه BIB می شود. از باز برای تولید نت های اخطار و ملودی استفاده می شود. ولتاژ نامی این باز 5 ولت است، جریان مصرفی آن کمتر از 30 میلی آمپر است و فرکانس تشدید آن 2300 500 هرتز است.

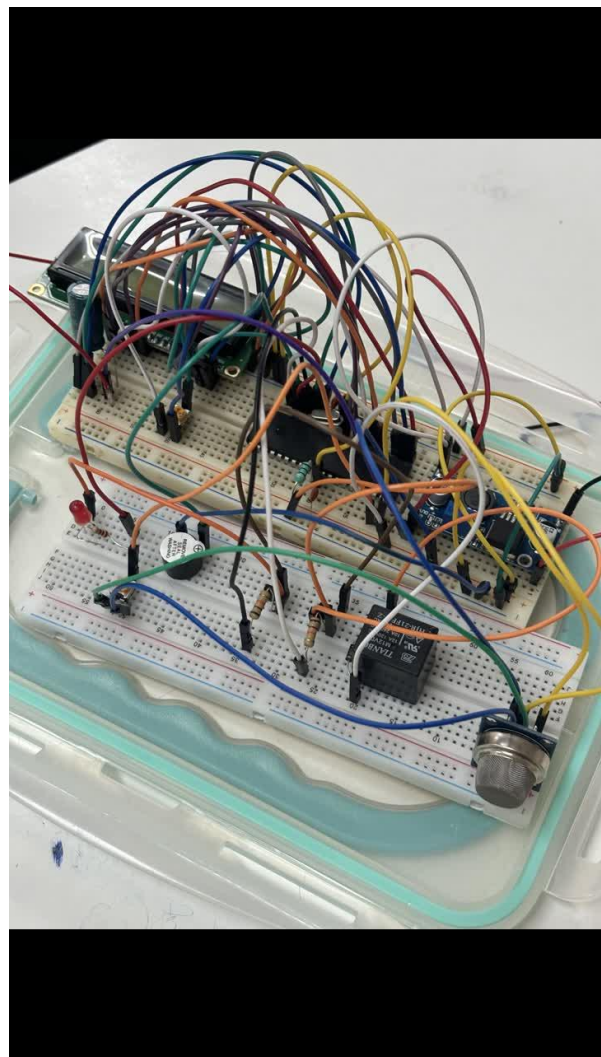
1-3- نقشه سخت افزار پروژه



در بخش سخت افزار پروژه ما برای اتصال پایه های میکرو به ماژول و مقدار خازن ، مقاومت و سلف از دیتاشیت محصولات استفاده کردیم و مدار را طبق رجیستر هایی که در ATMEGA16 تعریف کرده ایم متصل میکنیم.

به عنوان مثال در زمان ارسال پیامک توسط SIM800 ,مدار نیاز به ولتاژ زیادی دارد. به همین دلیل از یک خازن 1000 میکروفاراد استفاده کرده ایم تا ولتاژ مورد نیاز را در زمان ارسال پیامک تامین کند

1-4 نتایج عملی



طبق تصویر بالا مشاهده میکنید که غلظت گاز در محیط را به کمک LCD نمایش میدهیم. ما از رله 5 ولت به عنوان کلید الکتریکی استفاده کردیم تا وقتی که حساسیت غلظت گاز به بالای 580 میرسد کلید

قطع شود و صدای بازر فعال می شود و LED را روشن میکند و از طریق مازول SIM800 پیامک ارسال می شود و صاحب خانه را از نشت گاز مطلع میکند.

در مدار کاربرد پتانسیومتر 10K برای تغییر حساسیت مازول به گاز هست. به عنوان مثال ما هرچقدر میزان مقاومت را بیشتر کنیم حساسیت به گاز هم بیشتر می شود و عدد سریع تر به 580 میرسد.

فصل دوم

نرم افزار

1-2 توضیحات خط به خط کدها:

در ابتدا قبل از هرچیز، کلاک cpu را مشخص میکنیم و کتابخانه های مورد نیاز را فراخوانی میکنیم:

```
#define F_CPU 8000000UL
#include <util/delay.h>
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include "LCD.h"
```

سپس به تعیین پورت و پین های مورد نیاز برای ارتباط سریال با ماژول SIM-800 میپردازیم:

```
#define SIM800_TX_PORT PORTD
#define SIM800_TX_PIN PD1
#define SIM800_RX_PORT PORTD
#define SIM800_RX_PIN PD0
```

بعد برای کنترل ارسال پیامک در برنامه فلگ تعریف میکنیم (به این صورت که هر موقع فلگ ما یک شد،

GSM پیامک ارسال کند). در این پروژه BUADRATE را 9600 در نظر میگیریم و سپس به تعریف شماره

موبایل کاربر و پیام های مورد نظرمون میپردازیم :

```
volatile uint8_t smsFlag = 1; // Initialize flag to 1 initially
unsigned int BuadRate = 9600;
char buffer[20];
char Phone[] = "+989918726450"; // Replace with the recipient's phone number
char GasMessage[] = "AirQulity is Bad";
char NoGasMessage[] = "AirQulity is Good";
```

حال تابع راه اندازی ADC را مینویسیم:

```
void ADC_Init(){
    DDRA = 0x00; // Set PORTA as input
    ADCSRA = 0x87; // Enable ADC, set prescaler to 128
    ADMUX = 0x40; // AVCC with external capacitor at AREF pin
}
```

ابتدا پورت A را به عنوان ورودی تعیین میکنیم. سپس ADC را با پیش تقسیم کننده 128 فعال میکنیم. در آخر ورودی مرجع را AVCC با خازن متصل در AREF تنظیم میکنیم.

تابع خواندن مقدار ADC :

```
int ADC_Read(char channel){
    int A, ALow;
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F); // Select ADC channel
    ADCSRA |= (1 << ADSC); // Start conversion
    while(!(ADCSRA & (1 << ADIF))); // Wait for conversion to complete
    _delay_us(10);

    ALow = (int)ADCL;
    A = (int)ADCH * 256;
    A = A + ALow;

    return A;
}
```

با استفاده از این تابع مقدار ADC از کانال مورد نظرمان را میخوانیم و آنرا در تابع اصلی فراخوانی میکنیم.

دستورات مربوط به ارسال پیامک به وسیله GSM :

```
void GSM_SendSMS(char* phone, char* message, int Air) {
    // Send AT command to set SMS mode
    LCD_Clear();
    LCD_String_xy(0, 1, "Set GSM");
    send_AT_command("AT+CMGF=1\r");
    _delay_ms(1000); // Wait for response

    // Send AT command to set recipient phone number
    LCD_Clear();
    LCD_String_xy(0, 4, "Phone: ");
    LCD_String_xy(1, 3, phone);
    char command[30];
    sprintf(command, "AT+CMGS=\"%s\"\r", phone);
    send_AT_command(command);
    _delay_ms(1000); // Wait for response
}
```

```

// Send the SMS message
LCD_Clear();
LCD_String_xy(0, 4, "Send SMS");
send_AT_command(message);
_delay_ms(100); // Wait for response

// Send Air Quality
LCD_Clear();
LCD_String_xy(0, 1, "Send Data");
char Data[30];
sprintf(Data, "AirQuality: %d", Air);
send_AT_command(Data);

_delay_ms(500);

// Send Ctrl+Z to indicate the end of the message
LCD_Clear();
LCD_String_xy(0, 1, "Done");
UART_Transmit(0x1A);
_delay_ms(100); // Wait for response

// Wait for SMS to be sent
_delay_ms(500);
smsFlag = 0; // Reset the flag after sending SMS
}

```

حال تابع راه اندازی USART برای ارتباط سریال را مینویسیم:

```

void UART_Init(unsigned int baud_rate) {
    // Set baud rate
    unsigned int ubrr = F_CPU / 16 / baud_rate - 1;
    UBRRH = (unsigned char)(ubrr >> 8);
    UBRRL = (unsigned char)ubrr;

    // Enable receiver and transmitter
    UCSRB = (1 << RXEN) | (1 << TXEN);

    // Set frame format: 8 data bits, 1 stop bit, no parity

```

```
UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
}
```

که در آن BUADRATE و تنظیمات مربوط به ارسال پیامک را پیکربندی میکنیم.

تابع ارسال داده از طریق UART :

```
void UART_Transmit(unsigned char data) {
    // Wait for empty transmit buffer
    while (!(UCSRA & (1 << UDRE)));

    // Put data into buffer, sends the data
    UDR = data;
}
```

برای خالی شدن بافر ارسال منتظر می مانیم و دیتا را در بافر USART قرار میدهیم.

تابع مربوط به دستورات AT :

```
void send_AT_command(const char *command) {
    // Transmit each character of the command
    while (*command) {
        UART_Transmit(*command);
        command++;
    }

    // Send carriage return and line feed
    UART_Transmit('\r');
    UART_Transmit('\n');
}
```

در ادامه تایمر 1 را راه اندازی میکنیم:

```
void Timer_init() {
    // Set Timer1 in normal mode with scale 1024
    TCCR1A = 0;
    TCCR1B = (1 << CS12) | (1 << CS10);
    // Enable Timer1 overflow interrupt
    TIMSK |= (1 << TOIE1);
    // Set Timer1 value for a 1-minute interval
    TCNT1 = 65535 - (F_CPU / 61440); // Adjust based on your MCU frequency
    // take 7.5 sec to overflow Timer1
}
```


تا با استفاده از وقفه، روال یک شدن فلگ و ارسال پیامک یک دقیقه طول بکشد. به این ترتیب هر یک دقیقه برای کاربر پیامک ارسال خواهد شد:

```
ISR(TIMER1_OVF_vect){
    static uint16_t timer_count = 0;
    timer_count++;
    if(timer_count >= 8) { // 8 * 7.5 = 1 minute
        smsFlag = 1;
        timer_count = 0;
    }
}
```

حال که تنظیمات مربوط به ADC و ارتباط سریال را انجام داده ایم و ماژول GSM راه اندازی شد، تابع اصلی مورد نظر را مینویسیم:

```
int main(void)
{
    // Set USART
    UCSRC = 0x86; //0b10000110
    // UCSZ1 & UCSZ0 == we use 8-bit
    UBRRH = 0x00; //0b00000000
    UBRRL = 0x06; //0b00000110
    DDRA = 0x00; // Set PORTA as input
    DDRD = 0b11111100; // Set PORTD as output except PD0 and PD1
    UART_Init(BuadRate);
    Timer_init();
    ADC_Init();
    LCD_Init();
    sei();
    int AirValue;
    int AirSensor;
    int Pot_Value;
    // Welcome
    LCD_Clear();
    LCD_String_xy(0, 4, "Welcome");
    _delay_ms(1000);

    while (1)
    {
        AirValue = ADC_Read(0);
        Pot_Value = ADC_Read(1);
        float AirBuffer;
```

```

_delay_ms(100);
if (Pot_Value < 128 )
{
    AirBuffer = AirValue * 0.125;
}
else if (Pot_Value < 256)
{
    AirBuffer = AirValue * 2 / 8 ;
}
else if (Pot_Value < 384)
{
    AirBuffer = AirValue * 3 / 8 ;
}
else if (Pot_Value < 512)
{
    AirBuffer = AirValue * 4 / 8 ;
}
else if (Pot_Value < 640)
{
    AirBuffer = AirValue * 5 / 8 ;
}
else if (Pot_Value < 768)
{
    AirBuffer = AirValue * 6 / 8 ;
}
else if (Pot_Value < 896)
{
    AirBuffer = AirValue * 7 / 8 ;
}
else if (Pot_Value < 1024)
{
    AirBuffer = AirValue * 8 / 8 ;
}
_delay_ms(100);
//AirBuffer = 500.55;
AirSensor = (int)AirBuffer;
// Show AirQuality
if (AirSensor > 580)
{
    LCD_Clear();
    sprintf(buffer, "AirQuality: %d", AirSensor);
}

```

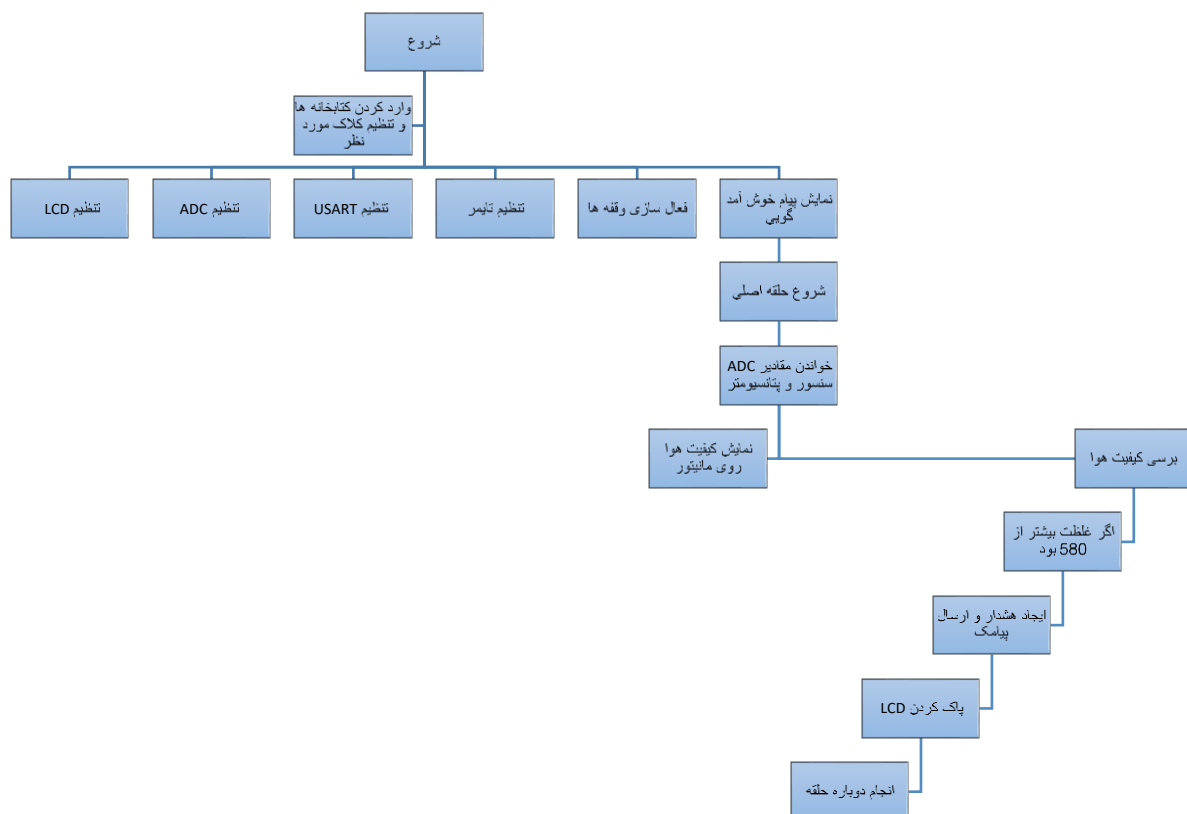
```

        LCD_String_xy(0, 1, buffer);
        LCD_String_xy(1, 4, "Gas");
        PORTD |= (1 << 2);
        PORTD |= (1 << 3);
        PORTD |= (1 << 4);
    }
    else
    {
        LCD_Clear();
        sprintf(buffer, "AirQuality: %d", AirSensor);
        LCD_String_xy(0, 1, buffer);
        LCD_String_xy(1, 4, "NoGas");
        PORTD &= ~(1 << 2);
        PORTD &= ~(1 << 3);
        PORTD &= ~(1 << 4);
    }
    // Send SMS
    if (smsFlag == 1)
    {
        _delay_us(50);
        if (AirSensor > 580)
        {
            GSM_SendSMS(Phone, GasMessage, AirSensor);
        }
        else
        {
            GSM_SendSMS(Phone, NoGasMessage, AirSensor);
        }
        smsFlag = 0; // Reset the flag after sending SMS
    }
    _delay_ms(250);
    LCD_Clear();
}
}

```

تابع اصلی ما هر موقع مقدار گاز موجود در هوا بیشتر از 580 بود فلگ را یک میکند تا پیامک به کاربر ارسال شود. پتانسیومتر به هشت بازه تقسیم شده تا ضرب آن در مقدار اعلامی ماژول مقدار غلظت را برای ما مانیتور کند. (به این صورت که مقدار پیش پتانسیومتر ما را در یکی از این بازه ها قرار میدهد به طوریکه مثلا اگر پتانسیومتر را تا انتها بچرخانیم در آخرین بازه قرار میگیریم و حساسیت ما بیشتر می شود).

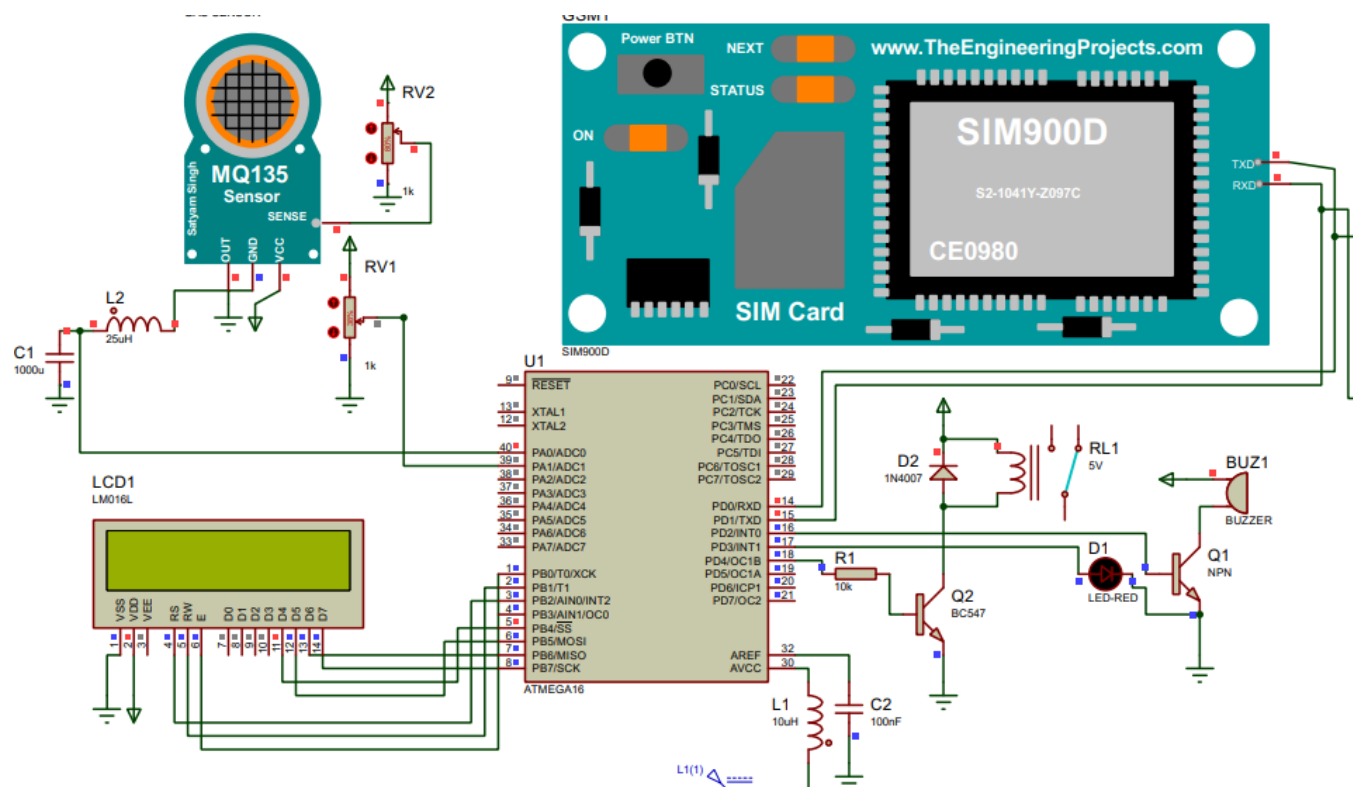
2-2 فلوچارت برنامه



فصل سوم

نتایج

3-1 نتایج شبیه سازی در نرم افزار پروتئوس:



2-3 مراجع

1. ویدیو ها و دوره های آموزشی در <http://www.youtube.com>
2. ویدیو های آموزشی وبسایت آرينکس [/https://irenx.ir](https://irenx.ir)
3. کتاب میکرو کنترلر های AVR تالیف جابر الوندی